

dydi

¿Aguanta la capa gratuita?

Evaluación de una arquitectura de microservicios en tiempo real sobre el *free tier* de Render, bajo inyección de carga.

Equipo 3 · García Páez David Israel · Solís Flores Irvin Alfonso · Cervantes Guerrero Keila Yuridia · Casiano Gamzi Juan David
Universidad Tecnológica de Durango · Integradora 2026

EL PROBLEMA

Los proyectos de la carrera casi nunca llegan a producción.

Porque un servidor cuesta. Las capas gratuitas existen, pero tienen fama de servir solo para demos.

512

MB de RAM por servicio

15

minutos y el servicio se duerme

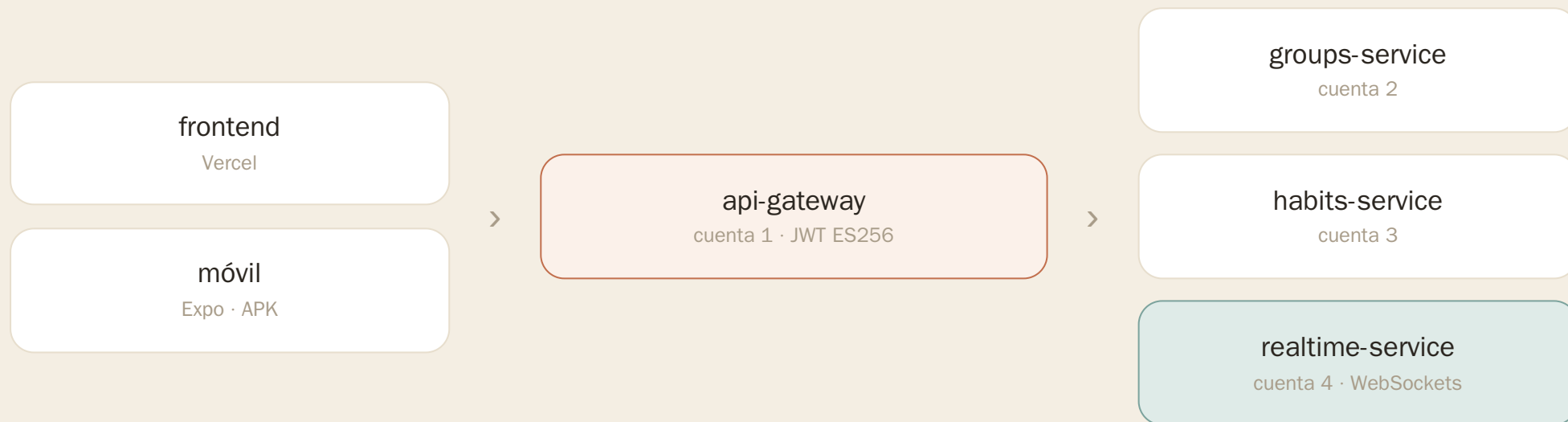
\$0

presupuesto disponible

Esa fama estaba repetida, pero nadie la había medido.

Dydi: cuatro servicios en Go, cuatro cuentas gratuitas.

App real de *accountability* social: grupos de ≤ 8 amigos, hábitos diarios y ruleta de penitencias el sábado, transmitida en vivo.



Una cuenta gratuita por servicio, aportadas por cuatro integrantes: el sistema se compone con los recursos que legítimamente teníamos.

LA APUESTA METODOLÓGICA

El tiempo real lo programamos nosotros. A propósito.

PUDIMOS DELEGARLO

Supabase Realtime resolvía la pieza en una tarde.

Y ENTONCES NO MEDIMOS NADA

Queremos saber cuánto cuesta sostener conexiones vivas dentro de 512 MB. Si la delegamos, esa pieza sale del sistema bajo prueba.

Es la variable central del experimento.

MÉTODO

Dos fuentes independientes que se cruzan.

POR FUERA · K6

Rampas de 100, 1 000, 2 500 y 5 000 usuarios virtuales. 3 repeticiones por nivel, misma ventana horaria: un mal día no es un resultado.

POR DENTRO · TELEMETRÍA PROPIA

~200 líneas de Go por servicio: memoria, latencia por ruta y estado del *pool* de base de datos, muestreadas cada 5 s.

Cada corrida deja cuatro artefactos, incluido el commit exacto del código medido. Y una regla: si un servicio muere, el hueco en la serie también es dato.

El piloto que casi nos engaña.

	PILOTO 1	PILOTO 2
Peticiones fallidas	88.30 %	0.00 %
Conexiones rechazadas	93.68 %	0.00 %
RAM observada	44 MB	44 MB

El sistema estaba *aburrido* mientras el cliente reportaba un desastre. Ahí estaba la pista.

LA CAUSA

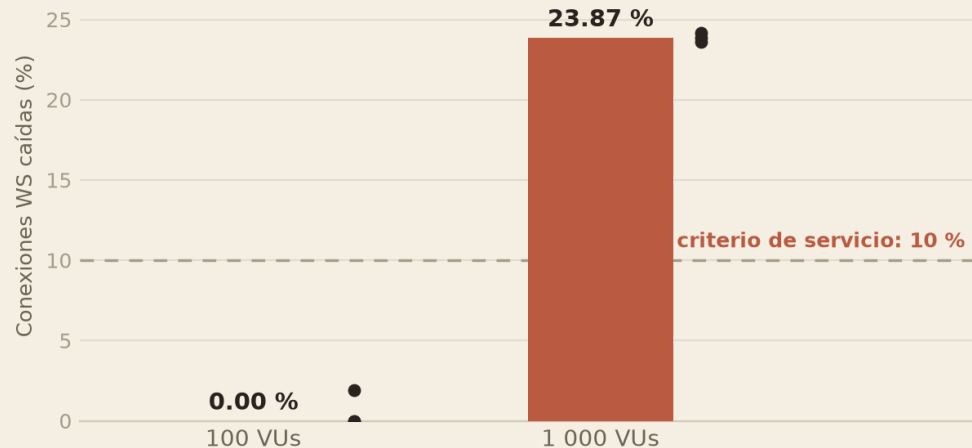
Nuestro propio gateway limita 5 peticiones/s por usuario, y todos los usuarios virtuales compartían una misma cuenta de prueba.

Estábamos midiendo el limitador, no la arquitectura.

Sin telemetría del lado del servidor, habríamos publicado una mentira con gráficas bonitas.

Conexiones WebSocket caídas por nivel de carga

Barra = mediana de 3 repeticiones · puntos = repeticiones individuales (Sesión 1)



23.87%

de las conexiones en vivo
no logran establecerse a 1 000 usuarios

UMBRAL PREDEFINIDO: 10 %

Las tres repeticiones dieron 23.59, 23.87 y 24.16 %. Con esa consistencia, el ruido de plataforma queda descartado como explicación. Y no son sesiones que se corten a medio vuelo: todas son rechazos en el handshake, antes de que el socket exista.

El plano HTTP, en cambio, no falla: 0 errores en 64 706 peticiones, solo 26 % más lento.

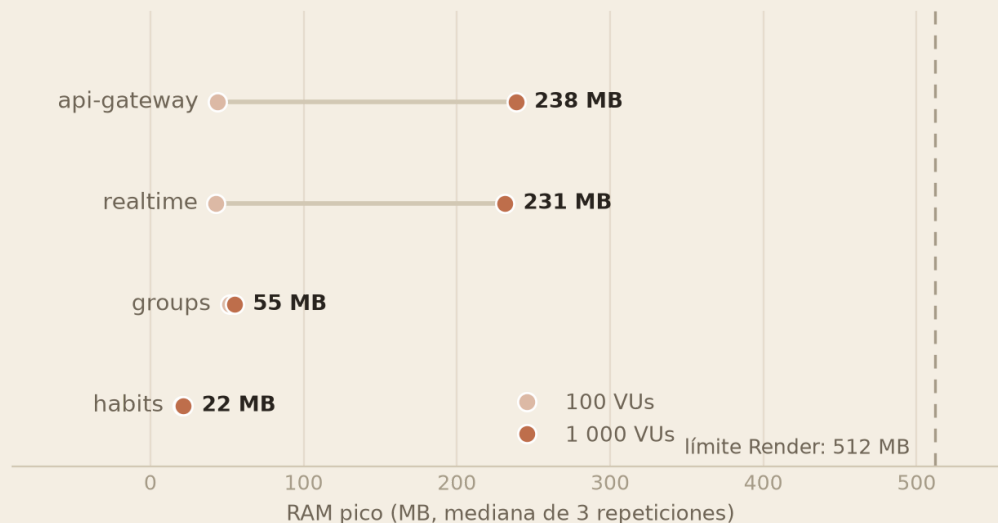
Falló con la mitad de la memoria libre.

46.6%

del límite de 512 MB en el servicio más cargado, en el momento del incumplimiento

Nuestra hipótesis apuntaba a un agotamiento de memoria. El sistema falló por otra vía.

RAM pico por servicio: 100 → 1 000 VUs



El costo se concentra en gateway y realtime (×5); los servicios transaccionales permanecen planos. Todo el espacio a la derecha es memoria que sobraba.

EL MECANISMO

Se rompió por cómo están amarradas las piezas.

1 · Conexión

Alguien abre un WebSocket

2 · Verificación

realtime pregunta a groups:
«¿pertenece a esta sala?»

3 · Congestión

Esa ruta toca la base de datos
y se satura

4 · Tormenta

Se rechazan conexiones →
reintentos → más handshakes

Es una decisión de seguridad correcta, porque cierra el sistema por defecto, pero ata la salud del canal en vivo a la latencia de un servicio transaccional. Cada reintento cae sobre un gateway que ya estaba al 100 % de su CPU (0.1 vCPU).

RESULTADO INESPERADO

La factura del *free tier*.

Las reglas de operación pesan tanto como los límites de cómputo, y tumbaron a nuestro propio experimento.

17.8 GB

movidos contra una cuota de
5 GB/mes. Render
suspendió dos cuentas

8 días

y Supabase pausó la base:
todo respondía errores
estando sano

**RAM ▲
CPU ▼**

firma medida: memoria al
tope con la CPU ociosa.
Causa más probable, no
confirmada: créditos de CPU
de la base agotados

–89.6 %

de tráfico al activar
compresión. Medido, no
estimado

La capa gratuita te limita por cuotas antes que por CPU.

¿Y EN USUARIOS REALES?

~3 200

usuarios activos diarios ($\approx$ 400 grupos llenos)

ESTIMACIÓN DECLARADA

Ley de Little aplicada sobre la concurrencia que sí medimos, con supuestos declarados: sesiones de ~5 min, ~1.5 al día, 25 % en hora pico.

Por un camino independiente, la cuota mensual de transferencia con compresión, salen entre 1 100 y 3 700. Que dos límites distintos converjan en el mismo orden de magnitud da confianza.

Aun así es una estimación con supuestos declarados, no un resultado experimental.

CONCLUSIONES

Sí aguanta, con límite medido.

LO QUE PODEMOS AFIRMAR

- ▮ 100 conexiones concurrentes con holgura amplia en memoria y estabilidad, y ninguna en latencia
- ▮ A 1 000, el canal en vivo cruza su umbral: 23.87 % de caídas
- ▮ El quiebre llegó por acoplamiento entre servicios
- ▮ Las cuotas operativas condicionan la viabilidad

LOS LÍMITES DE LO MEDIDO

- ✗ No corrimos 2 500 ni 5 000: la propia capa gratuita agotó la ventana
- ✗ Todos los usuarios virtuales usaron una sola cuenta
- ✗ El inyector fue una sola máquina y una sola red
- ✗ Es un caso (Go + Render + Supabase), no todo *free tier*

CIERRE

Todo es reproducible.

El arnés completo, los datos crudos de las once corridas con su *commit*, la bitácora y el script que regenera banco, estadísticas y figuras con un solo comando.

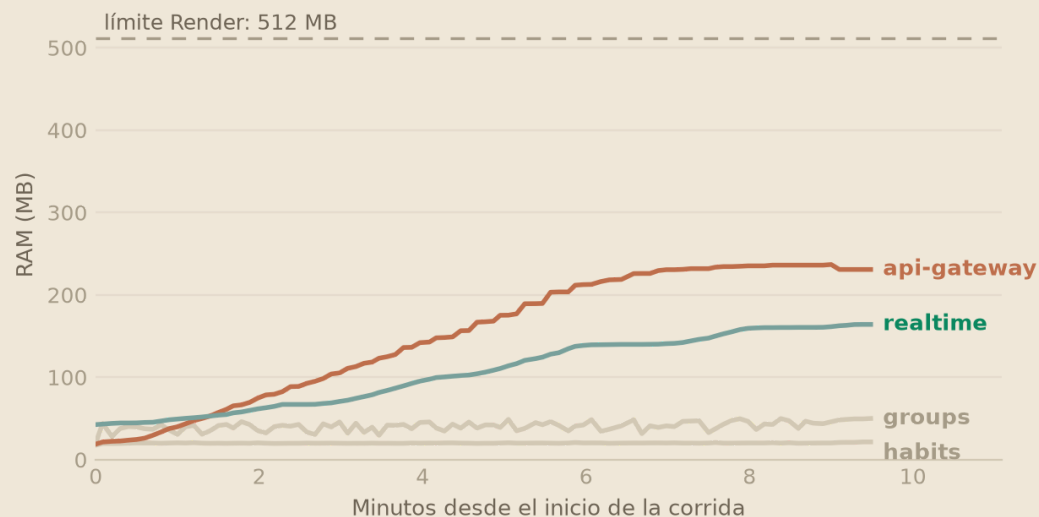
REPOSITORIO

github.com/davidgarcia57/Dydi

Gracias. ¿Preguntas?

RAM durante la corrida de 1 000 VUs (rep. 1)

Muestreo cada 5 s vía /metrics · la rampa de conexiones dura ~9 min



¿Hay fuga de memoria?

No. El consumo sigue a la rampa de conexiones y se aplana en la meseta: hay techo de carga, no fuga.

Por eso la proyección del primer OOM se sitúa entre 2 500 y 5 000 VUs, y por eso la llamamos *proyección*.

Mediana de 3 repeticiones (mín–máx)

MÉTRICA	100 VUS	1 000 VUS
Peticiones HTTP fallidas	0.00 %	0.00 % (0/64 706)
P95 HTTP (límite 800 ms)	826 ms (790–847)	1 045 ms (955–1 138)
Conexiones WS caídas (límite 10 %)	0.00 % (0–1.92)	23.87 % (23.59–24.16)
P95 establecimiento WS (límite 2 s)	918 ms (894–932)	19 974 ms (19 817–20 251)
Verificaciones correctas (límite > 95 %)	100.00 %	95.67 % (95.60–95.74)
RAM pico api-gateway	43.6 MB	238.5 MB
RAM pico realtime	42.8 MB	231.3 MB
RAM pico groups	51.6 MB	54.8 MB
RAM pico habits	20.8 MB	21.5 MB

Los seis criterios fijados antes de medir: latencia HTTP P95 < 800 ms · P99 < 2 000 ms · fallos HTTP < 5 % · establecimiento WS P95 < 2 s · caídas WS < 10 % · verificaciones > 95 %. En rojo, los incumplidos. Las verificaciones cumplen, pero por 0.6 puntos y sin aportar evidencia independiente: sus fallas son las mismas del

¿Y si fue casualidad?

VARIABLE	CV A 100	CV A 1 000
Conexiones WS caídas	173.4 %	1.2 %
P95 establecimiento WS	2.1 %	1.1 %
P95 HTTP	3.5 %	8.8 %
RAM pico api-gateway	0.3 %	0.5 %
RAM pico realtime	16.3 %	28.1 %

Coefficiente de variación entre las tres repeticiones. El 173.4 % es artefacto: la media ronda cero (0, 0 y 1.92 %).

La conclusión descansa en la variable más estable de todas: las caídas a 1 000 VUs, con un CV de 1.2 %.

La menos estable es la RAM de realtime (28.1 %), y por eso sólo se usa para el patrón grueso —el crecimiento $\times 5$ — y nunca para una proyección puntual.

Dentro de una ventana horaria el sistema es casi determinista. Entre ventanas distintas, no: el mismo nivel de 100 dio 826 ms y 404 ms en dos sesiones. Por eso se reporta mediana con rango y no corridas únicas.